>_ The Ultimate Master Mission Report

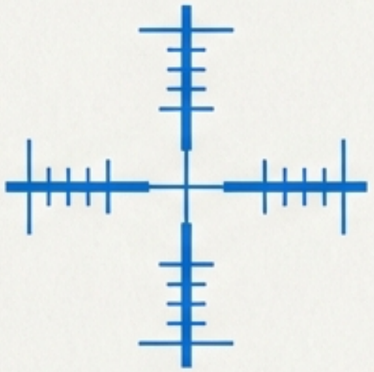
PropTech, Logistics, & R&D Synthesis

AUTHORIZATION: JOE'S BROTHER // STATUS: CLASSIFIED DEBRIEF

NotebookLM

THE MASTER MISSION OBJECTIVES

>_



TARGET 01: DIGITIZE PROPTECH

Modernize user interfaces with rapid, vision-enabled iteration.

>_

>_

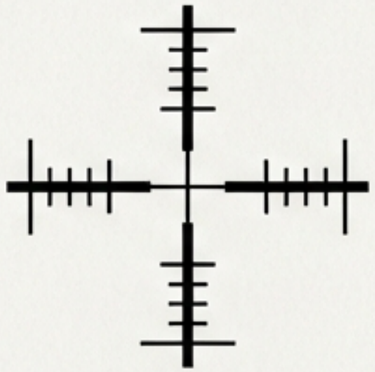


TARGET 02: OPTIMIZE LOGISTICS

Bulletproof routing and real-time asset tracking with zero data latency.

>_

>_



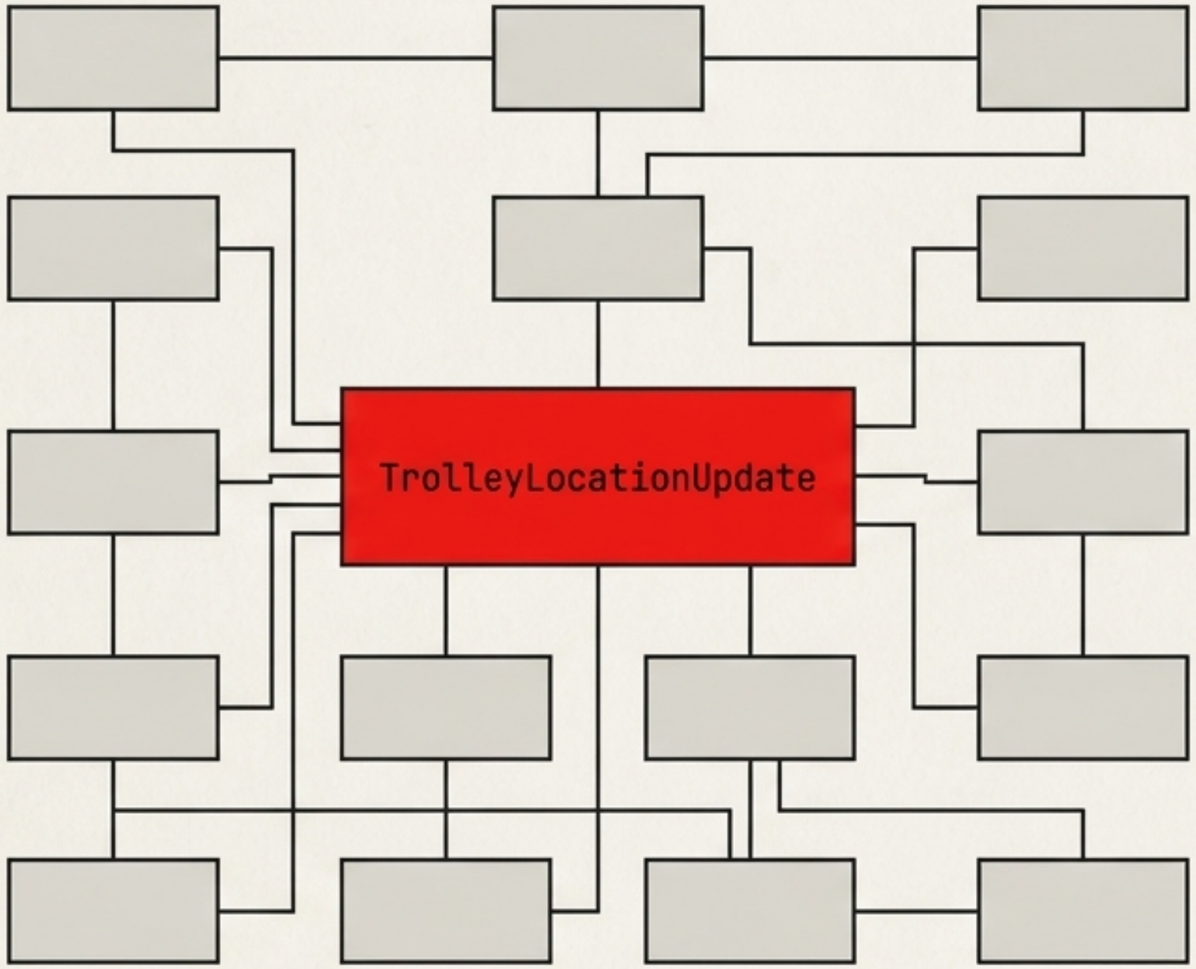
TARGET 03: THE CATALYST

Supercharge R&D to force-multiply the engineering output of Targets 01 and 02. Make it happen yesterday.

>_

NotebookLM

The Bottleneck: TrolleyLocationUpdate



[SYSTEM_STATUS]: Logistics Microservice requires a real-time endpoint for Trolley Location.

[TRADITIONAL_ESTIMATE]: 2 weeks (Resource allocation, scaffolding, boilerplate, testing).

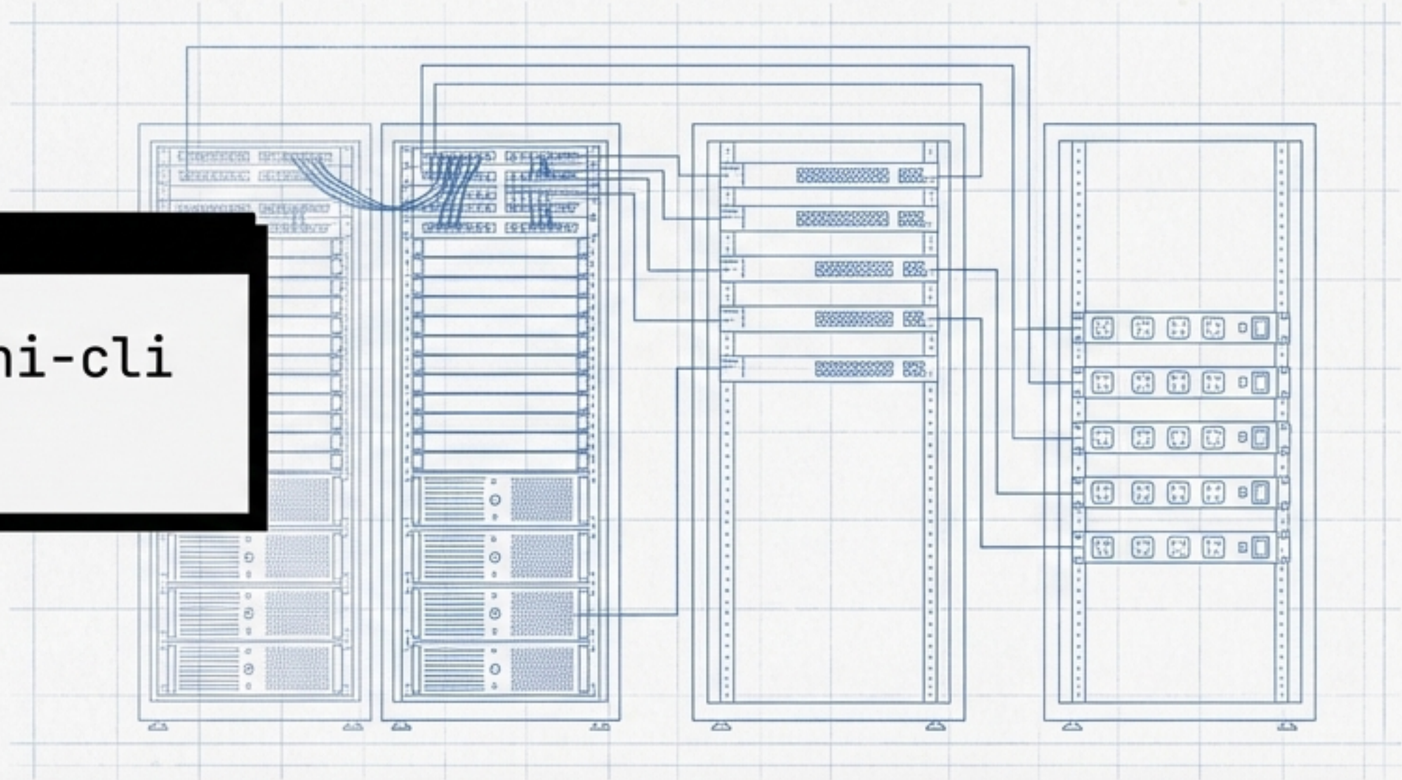
[R&D_VERDICT]: Unacceptable.

> We needed a surgical strike to implement this endpoint without derailing the PropTech sprint. We needed a new paradigm.

NotebookLM

Enter the Fixer: Google Gemini CLI

```
> npm install -g @google/gemini-cli
> gemini
```



Not a Chatbot: An open-source, terminal-native agentic coder.

Deep Integration: Lives natively in our local file system.

Action-Oriented: Doesn't just generate code snippets; reads files, executes shell commands, and builds workflows.

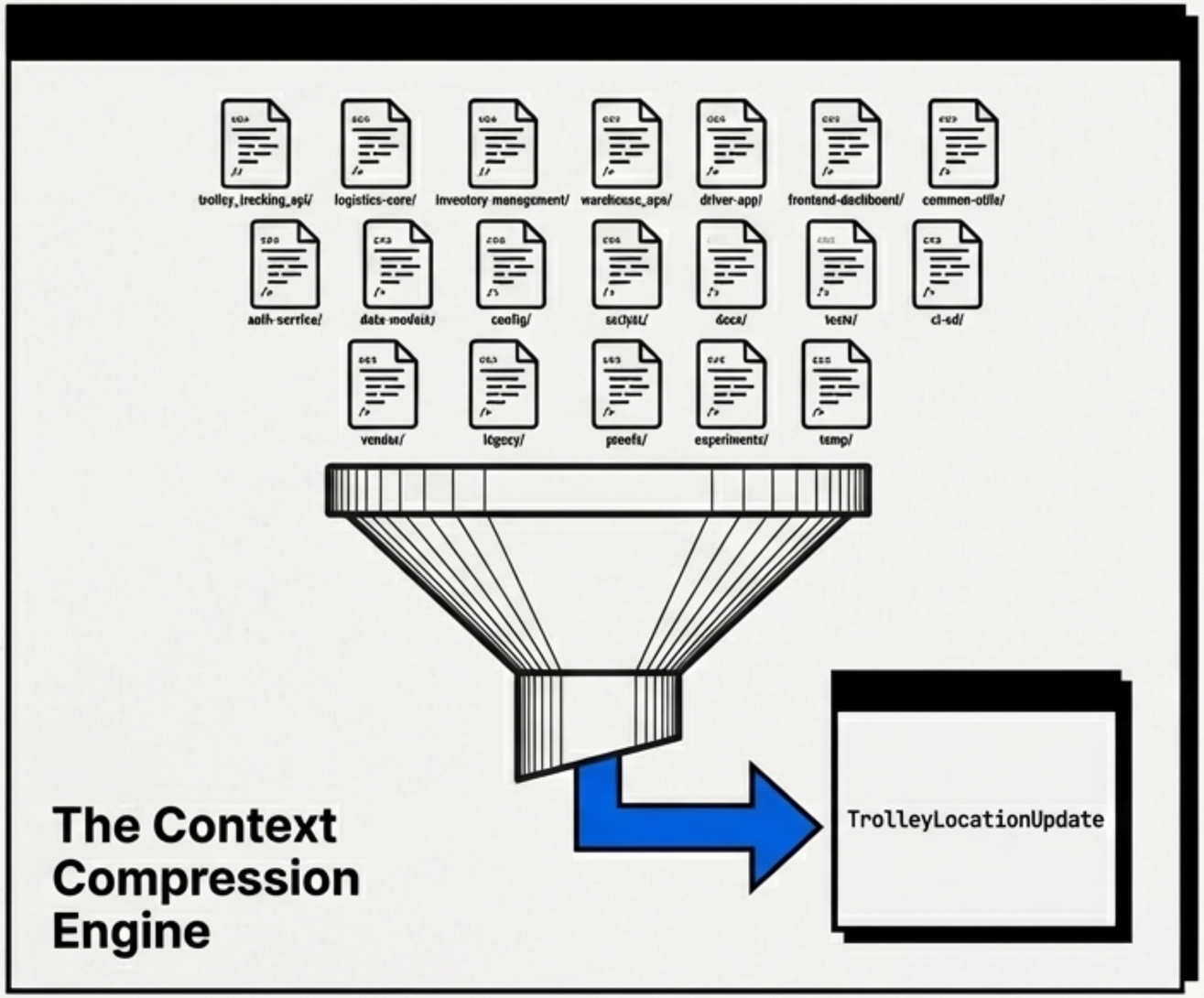
NotebookLM

Matrix 1: The Unfair Advantage Comparison

Capability	Standard Tools	**Gemini CLI**
Context Limits	32k - 200k tokens	**1,000,000 Tokens** (~1,500 pages of code)
Pricing	Pay-per-token overhead	**1,000 free requests/day** (Zero cost friction for R&D)
Execution	Manual copy/paste	**Autonomous Shell / File System Access**
Sandboxing	Unsecured	**Seatbelt (macOS) / Docker natively supported**

NotebookLM

Omniscient Repo Vision



[INGESTION]:

1M tokens means we didn't just show the agent the endpoint file. We fed it the entire Logistics monorepo.

[EXTERNAL_REACH]:

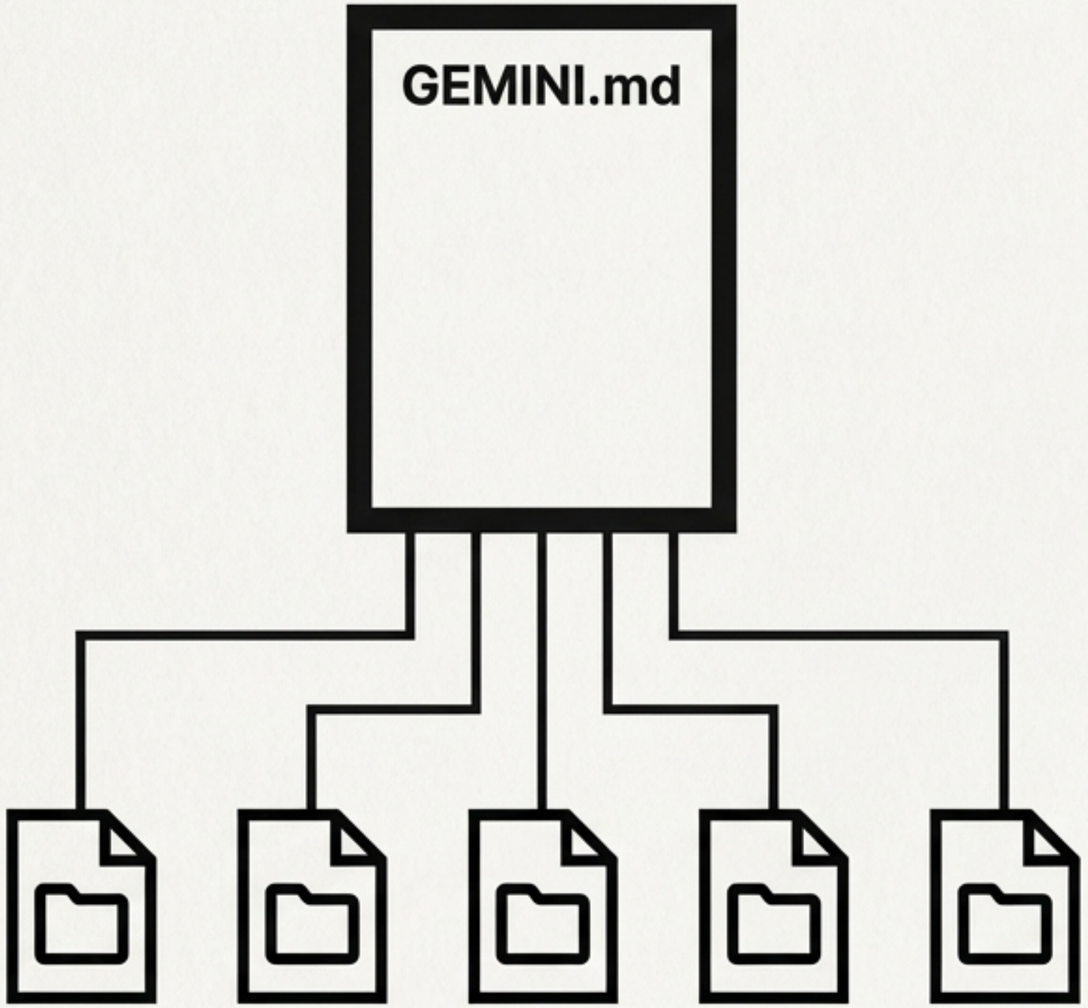
Using `/dir add ../PropTech`, we pulled in external UI dependencies without leaving the terminal.

[RESULT]:

Zero hallucinated variables.
Zero blind spots. Complete architectural awareness.

NotebookLM

The House Rules: GEMINI.md



Bootstrapped:
Generated instantly via the `/init` command.

Hierarchical Memory:
System rules -> Project rules -> Subdirectory rules.

The Mandates:
- **Stack:** Node.js, Firebase Realtime DB.
- **Standards:** Strict TypeScript, no `any` types.
- **Architecture:** Test-Driven Development is non-negotiable.

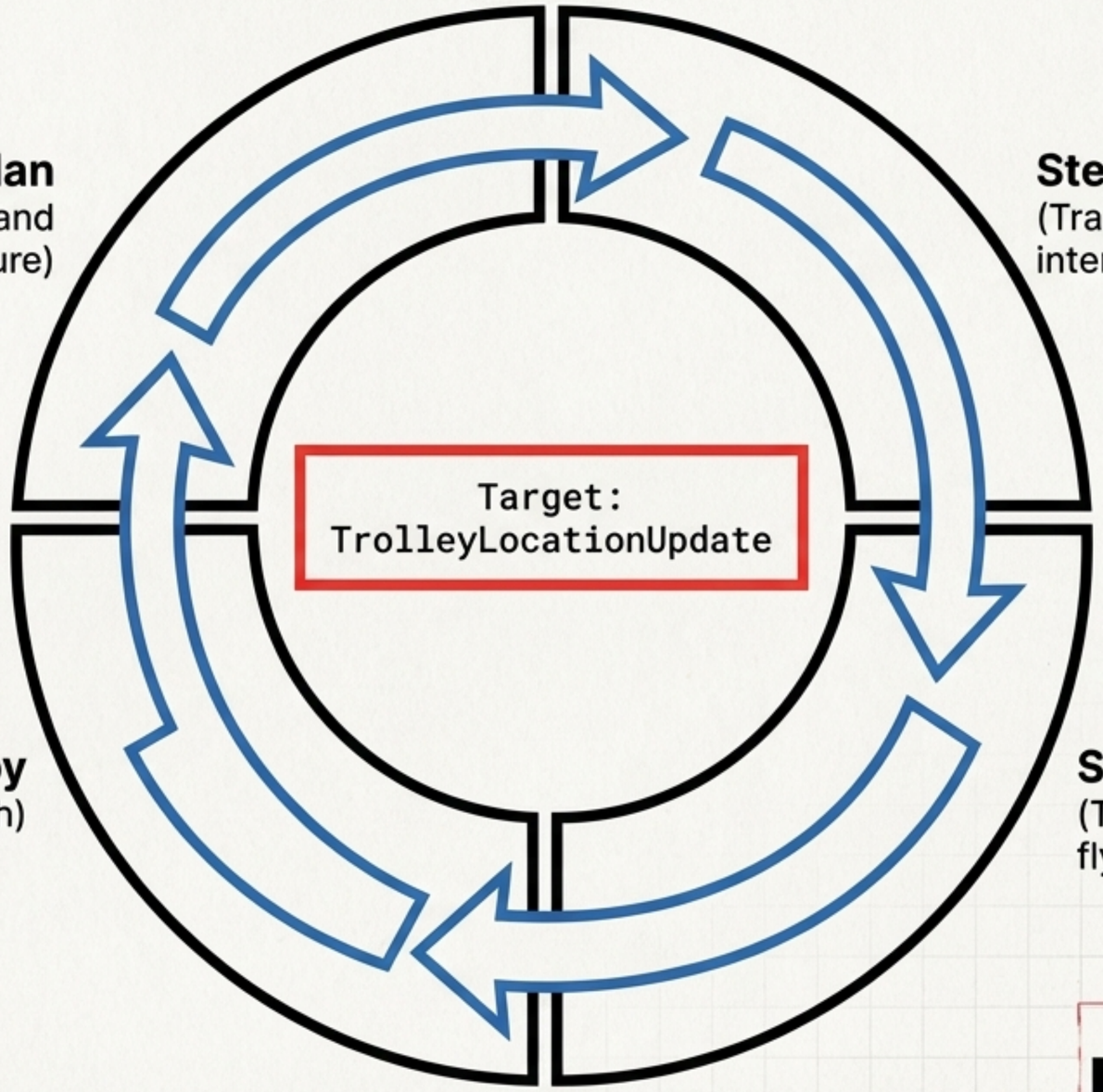
Bottom Line: It acts as a persistent project-level instruction set. It never forgets the rules of engagement.

NotebookLM

The Agentic Execution Loop

Step 1: Plan
(Ingest GEMINI.md and define architecture)

Step 2: Vibe Code
(Translate natural language intent into functional Node.js)

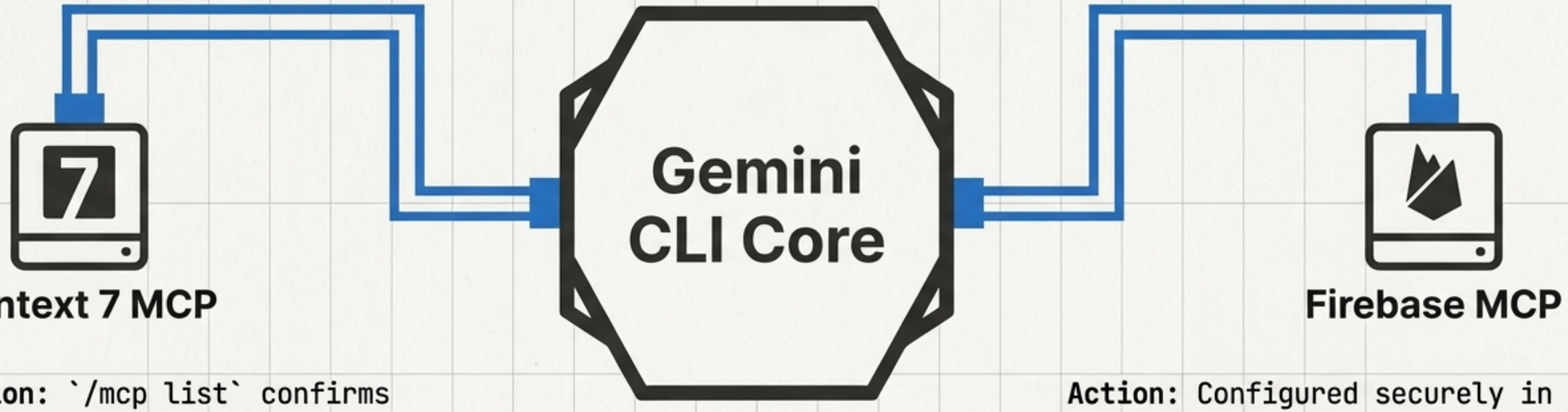


Step 4: Deploy
(Commit and push)

Step 3: TDD Loop
(Test-fail-implement flywheel)

NotebookLM

Step 1: Connecting the MCP (Model Context Protocol)



Action: `/mcp list` confirms active connection.

Purpose: Fetches the absolute latest Logistics API documentation directly from the source to prevent outdated syntax.

Action: Configured securely in `~/.gemini/settings.json`.

Purpose: Allows the agent to read our exact Realtime Database security rules before it writes a single line of logic.

NotebookLM

Step 2: The Custom Command Setup

.gemini/commands/logistics-build.toml

```
description = "Builds logistics endpoints against staged
changes"

prompt = "Review the following diff: {{ git diff staged
prompt = "Review the following diff: {{ git diff staged
}}. Apply the standards from @docs/guidelines.md."
```

Shell Injection:

Dynamically pulls the `git diff` straight into the prompt.

File Injection:

Uses the `@` syntax to force the agent to read specific PropTech/Logistics guidelines instantly.

NotebookLM

Step 3: Vibe Coding the Endpoint

INTENT (NATURAL LANGUAGE)

```
> /logistics-build "Create the
TrolleyLocationUpdate endpoint. Ensure the
trolley properties are strictly typed. Check
Firebase rules."

> _
```

OUTPUT (NODE.JS ARCHITECTURE)

```
const trolleyLocationUpdate = (req, res) => {
  const { trolley, locationData } = req.body;

  // Agent-Driven Validation Block
  if (!trolley || !locationData) return res.status(400);
  if (!trolley.properties) return res.status(400);

  // Firebase Rules Check & Write Operation
  // ... (Implementation details hidden)

  return res.status(200).send({ success: true });
};
```

Note: The agent autonomously identified and fixed an 'undefined properties' error before it ever hit staging.

NotebookLM

Step 4: The TDD (Test-Driven) Flywheel



1. Command

Instructed Gemini to use the `write_todos` tool to map out the test suite.

2. Red State

Agent builds the failing Jest tests for `TrolleyLocationUpdate`.

3. Execution

Agent autonomously refactors the controller logic.

4. Green State

Tests pass. Red, Green, Refactor—fully executed inside the CLI without breaking focus.

NotebookLM

Rules of Engagement: Operation Modes

Level 1: Default Mode (Interactive)	>_	**Action:** Requires explicit 'Y/N' for every file write and shell command. **Use Case:** Core architectural planning and safe exploration.
Level 2: Auto-Edit Mode (Shift+Tab)	>_	**Action:** Auto-approves code file edits, but halts for shell command execution. **Use Case:** Rapid iteration on the `TrolleyLocationUpdate` logic.
Level 3: YOLO Mode (—yolo / Ctrl+Y)	>_	**Action:** Full Auto. Keys to the motorcycle. Approves all tool calls and shell scripts. **Use Case:** Running the automated test suite and staging the final commits.

NotebookLM

Automated CI/CD Getaway



1

Headless Mode Integration:

By passing the `-p` flag and `--output-format json`, the Gemini CLI runs programmatically inside our GitHub Actions.

2

Pipeline Generation:

The CLI autonomously wrote the `cloudbuild.yaml` file required to containerize the endpoint.

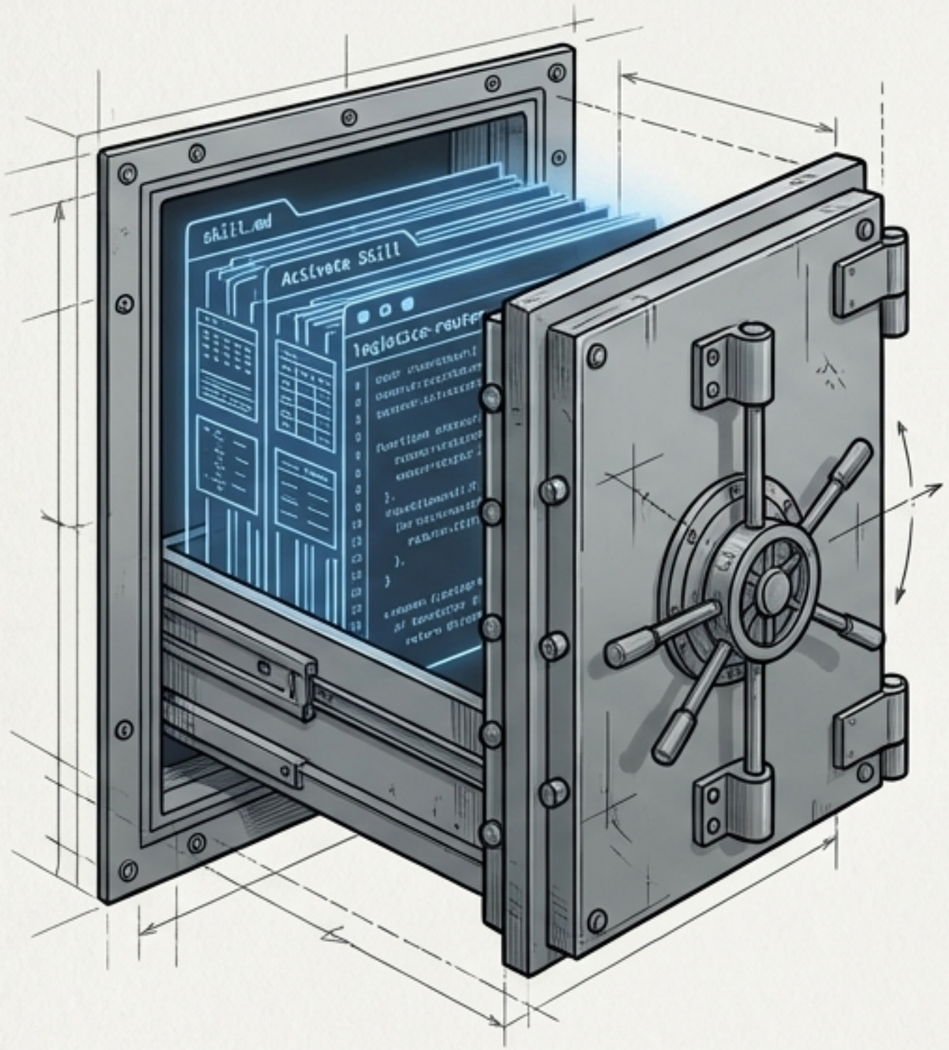
3

Status:

`TrolleyLocationUpdate` is live. Deployment time reduced from weeks to hours.

NotebookLM

Agent Skills (The R&D Future)



The Asset:

We didn't just build an endpoint. We extracted the Logistics routing logic and saved it as an Agent Skill (`skill.md`).

Progressive Disclosure:

Skills aren't loaded into context until activated via the `Activate Skill` tool. This preserves the 1M token window and saves massive compute costs.

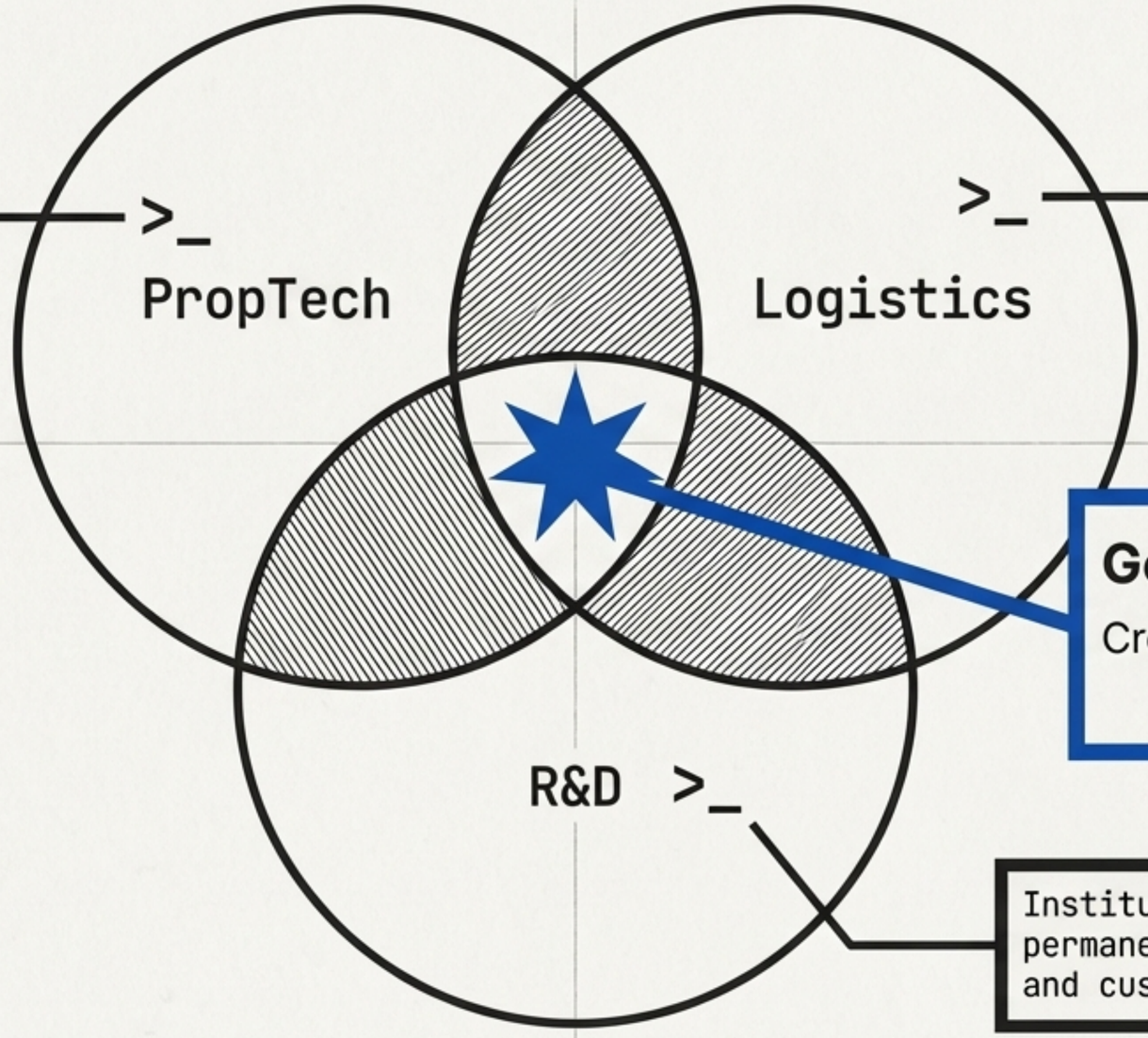
The Scale:

Now, any junior developer in PropTech can type `/activate logistics-router` and instantly code with the expertise of a senior logistics architect.

NotebookLM

Domain Synthesis: The Unified Model

Rapid UI iteration utilizing Multimodal Vision (Vibe coding UIs from `@mockup.jpg`).



Bulletproof data flow via MCP Servers & automated YOLO mode testing.

Gemini CLI Epicenter:

Cross-domain execution with zero context switching.

Institutional knowledge permanently captured in `GEMINI.md` and custom Agent Skills.

NotebookLM

> exit 0

[DEBRIEF_COMPLETE] : ENDPOINT_SHIPPED
[DOMAINS_UNIFIED]
[R&D_OPTIMIZED]

Joe's Brother, Out.

NotebookLM